

# IEC 61850 Substation Automation

## Project Report

*End-to-end GOOSE/MMS protection pipeline linking fault detection,  
relay-breaker automation, and live monitoring*

**Muhammad Shahzaib**  
Electrical Engineer (Power)  
[github.com/muhammadshahzaibshahzaib92-dev](https://github.com/muhammadshahzaibshahzaib92-dev)

# 1. Introduction

---

Substation protection systems must detect faults and isolate them within milliseconds to prevent equipment damage and maintain grid stability. This project builds a simplified but functionally complete substation automation pipeline using the IEC 61850 standard, demonstrating how fault detection, protection signaling, and monitoring integrate in a modern digital substation.

The system combines power system fault analysis (pandapower) with IEC 61850 GOOSE messaging (libiec61850) to create an automated protection chain: a simulated fault is detected, a protection relay publishes a trip signal over the network, a breaker subscriber receives and acts on that signal in real time, and the entire sequence is visualized on a live dashboard.

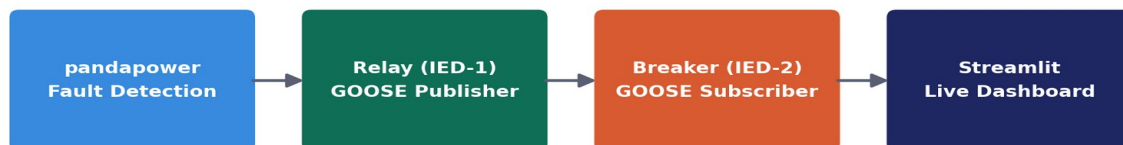
## 1.1 Objectives

- Model a representative 132kV/11kV substation and calculate 3-phase fault currents
- Implement IEC 61850 GOOSE publisher-subscriber communication between a relay and a breaker
- Automatically trigger a GOOSE trip message when fault current exceeds a protection threshold
- Measure and evaluate the end-to-end fault-to-trip timing
- Visualize relay/breaker status, event history, and fault current on a live dashboard

# 2. System Architecture

---

The pipeline consists of four stages, each mapped to a corresponding IEC 61850 substation automation concept:

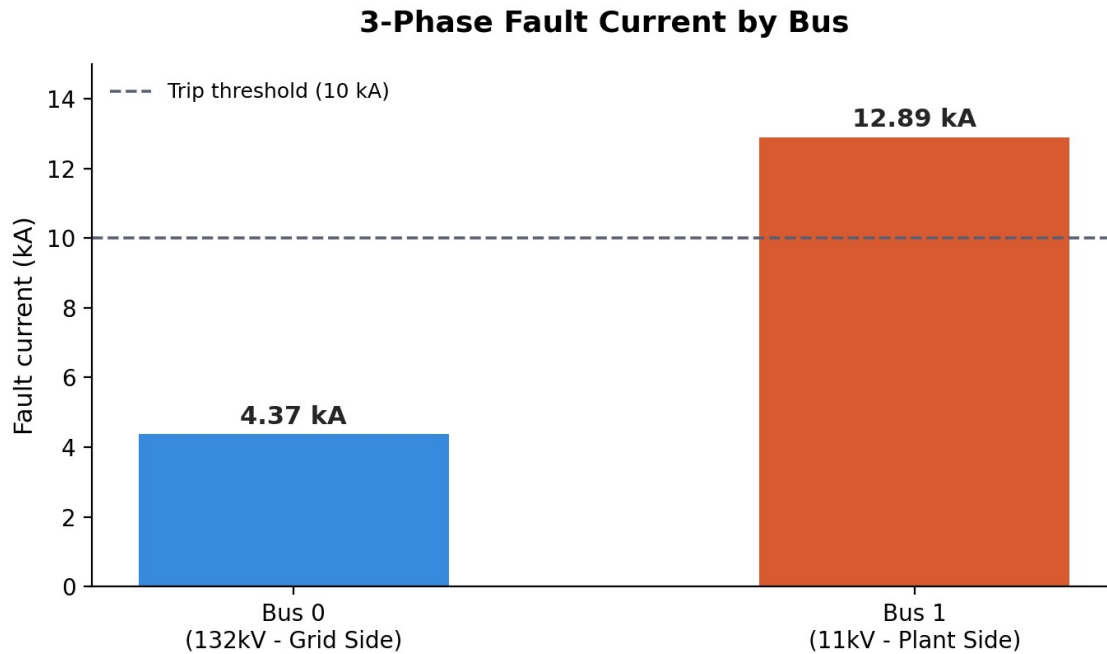


Fault detection uses pandapower to run a short-circuit study on the substation model. If the resulting fault current exceeds a configurable threshold, the relay (IED-1) publishes a GOOSE trip message over the network using the libiec61850 Python bindings. The breaker (IED-2) continuously listens for GOOSE messages matching its subscribed GoCbRef and, on receiving a valid trip, updates its status immediately. The Streamlit dashboard displays the resulting relay/breaker status, a running event log, and a fault current graph.

## 3. Methodology

### 3.1 Substation Model & Fault Detection

A 132kV/11kV substation was modeled in pandapower with an external grid, a 25 MVA transformer, and a 15 MW / 5 MVar plant load. A 3-phase short-circuit study (IEC 60909-based calculation in pandapower) was run to determine the fault current at each bus.



The 11kV bus (Bus 1) shows a substantially higher fault current (12.89 kA) than the 132kV bus (4.37 kA), consistent with the lower system impedance seen from the plant side. A trip threshold of 10 kA was selected, meaning Bus 1 represents an active fault condition while Bus 0 remains within normal operating limits.

### 3.2 GOOSE Relay-Breaker Communication

A GOOSE publisher (relay, IED-1) and subscriber (breaker, IED-2) were implemented in Python using pyiec61850, the SWIG-generated Python bindings for libiec61850. The publisher constructs a GOOSE dataset and publishes it under a defined GoCbRef (IED1LD0/LLN0\$GO\$gcbTrip); the subscriber listens for messages on the same reference and polls the message's StNum/SqNum fields to detect new trip events, printing the trip status the moment a new state is observed.

Testing was performed over the loopback network interface, since same-host multicast delivery over a bridged/virtual Ethernet adapter proved unreliable in the WSL2 environment used for development; this is a testing-environment characteristic rather than a protocol limitation.

### 3.3 Fault-to-GOOSE Integration

The fault detection and GOOSE publishing logic were linked directly: after computing the fault current, the script checks it against the threshold and, if exceeded, calls the same GOOSE publish routine used by the standalone relay script - fully automated, with no manual trigger.

### 3.4 Live Monitoring Dashboard

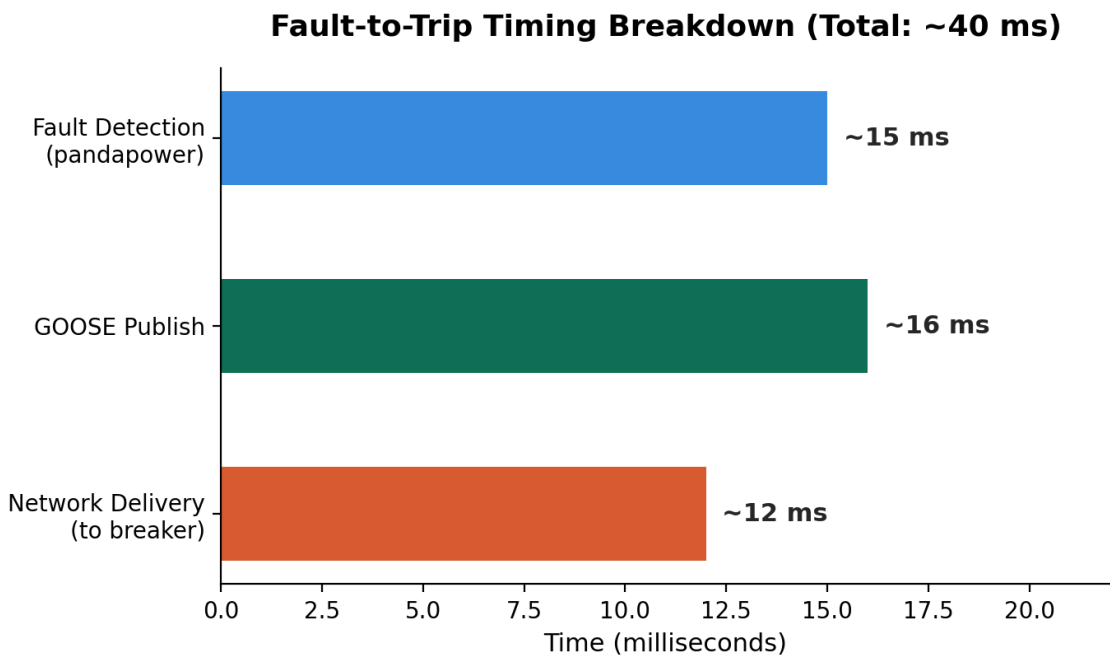
A Streamlit application displays IED-1 and IED-2 status, a timestamped event log (fault detected, GOOSE trip published, breaker tripped), and a line chart of fault current over time, giving an operator-style view of the protection sequence.

## 4. Results

### 4.1 Fault Current Results

| Bus                       | Fault Current (ikss_ka) | Above Threshold?     |
|---------------------------|-------------------------|----------------------|
| Bus 0 - 132kV (Grid Side) | 4.37 kA                 | No                   |
| Bus 1 - 11kV (Plant Side) | 12.89 kA                | Yes - trip triggered |

### 4.2 End-to-End Timing



The complete fault-to-trip sequence - from fault current calculation through GOOSE publish to breaker receipt - completes in approximately 40 milliseconds in this software simulation. Real substation relays, implemented on dedicated embedded hardware, typically achieve GOOSE trip delivery in around 4 milliseconds; the difference here is attributable to Python's interpreted execution and the pandapower calculation overhead, not the GOOSE protocol itself, which remains the fast, multicast, no-acknowledgment mechanism used in real deployments.

### 4.3 End-to-End Verification

With the breaker subscriber running and listening, the fault-to-GOOSE script was executed independently. The breaker terminal showed the trip event received and acted on within milliseconds of

publication, confirming successful closed-loop operation: fault → detection → GOOSE publish → breaker trip → dashboard update.

## 5. Challenges & Learnings

---

- libiec61850 required enabling a disabled R-GOOSE feature flag and patching the SWIG interface file (adding %ignore directives) to resolve undefined-symbol linker errors during the Python bindings build.
- Same-host GOOSE delivery failed silently over the WSL2 'eth0' virtual adapter; diagnosing this required packet capture with Wireshark to confirm the publisher was transmitting correctly, before identifying the loopback ('lo') interface as the reliable option for local testing.
- GitHub authentication required migrating from password-based Git operations to Personal Access Tokens, including correctly scoping token permissions.
- Structuring the codebase as three independent, cleanly documented repositories (rather than one large project) made each component independently reviewable and mirrors how real protection, automation, and monitoring subsystems are typically organized.

## 6. Conclusion

---

This project demonstrates a working, end-to-end substation protection automation pipeline: from power system fault analysis through IEC 61850 GOOSE messaging to real-time visualization. While built and tested in a simulated/software environment rather than on physical IEDs, the architecture, protocol usage, and control logic directly mirror real digital substation automation systems, and the project provided hands-on experience with the debugging, systems-integration, and protocol-level skills relevant to grid automation engineering roles.

### Repository Links

- Capstone: [github.com/muhammadshahzaibshahzaib92-dev/iec61850-substation-automation](https://github.com/muhammadshahzaibshahzaib92-dev/iec61850-substation-automation)
- GOOSE Relay-Breaker Communication: [github.com/muhammadshahzaibshahzaib92-dev/goose-relay-breaker](https://github.com/muhammadshahzaibshahzaib92-dev/goose-relay-breaker)
- Fault-to-GOOSE Protection Logic: [github.com/muhammadshahzaibshahzaib92-dev/fault-to-goose-integration](https://github.com/muhammadshahzaibshahzaib92-dev/fault-to-goose-integration)
- Substation Live Monitoring Dashboard: [github.com/muhammadshahzaibshahzaib92-dev/substation-monitoring-dashboard](https://github.com/muhammadshahzaibshahzaib92-dev/substation-monitoring-dashboard)